Task 14:

Design a simple microservices-based system by splitting a monolithic application into at least two independent services. Apply microservices design principles such as loose coupling, single responsibility, and independent deployment. Aim

Aim

To design and develop a **simple microservices-based system** by splitting a monolithic application into at least two independent services, applying principles such as **loose coupling, single responsibility, and independent deployment**.

Algorithm

1. Identify modules in monolithic app (e.g., Product & Order).
2. Split into two independent microservices:
    - **Product Service**
    - **Order Service**
3. Create separate Spring Boot projects for each service.
4. Define REST APIs for each service.
5. Ensure:
    - Each service has its own database (optional for demo).
    - Services are independent.
6. Run both services on different ports.
7. Test APIs individually using Postman.
8. Verify loose coupling and independent execution.

🧩 **Microservices Architecture**

- **Product Service** → Manages products
- **Order Service** → Manages orders

Code Implementation

- **1. Product Service**

Product.java

```
package com.productservice.entity;

import jakarta.persistence.*;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private double price;

    public Product() {}

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    // Getters & Setters
}
```

ProductRepository.java

```
package com.productservice.repository;

import com.productservice.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

ProductController.java

```
package com.productservice.controller;

import com.productservice.entity.Product;
import com.productservice.repository.ProductRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductRepository repo;

    public ProductController(ProductRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public List<Product> getAll() {
        return repo.findAll();
    }

    @PostMapping
    public Product add(@RequestBody Product product) {
        return repo.save(product);
    }
}
```

application.properties

```
server.port=8081
spring.datasource.url=jdbc:h2:mem:productdb
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

2. Order Service

📌 **Order.java**

```
package com.orderservice.entity;

import jakarta.persistence.*;

@Entity
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;
    private int quantity;

    public Order() {}

    public Order(String productName, int quantity) {
        this.productName = productName;
        this.quantity = quantity;
    }

    // Getters & Setters
}
```

OrderRepository.java

```
package com.orderservice.repository;
```

```
import com.orderservice.entity.Order;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {
}
```

OrderController.java

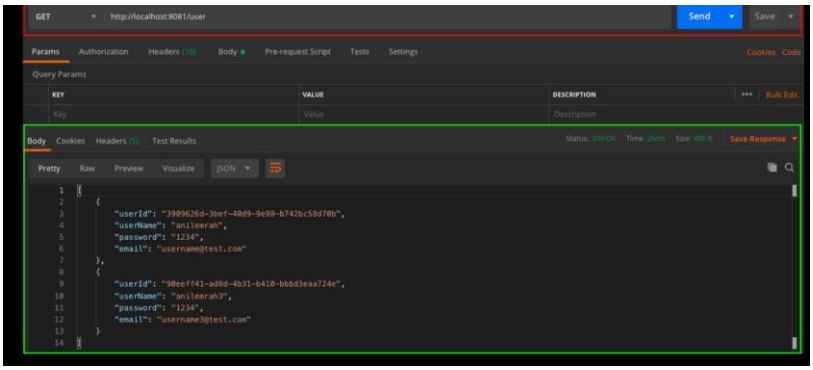
```
package com.orderservice.controller;

import com.orderservice.entity.Order;
import com.orderservice.repository.OrderRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderRepository repo;

    public OrderController(OrderRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public List<Order> getAll() {
        return repo.findAll();
    }

    @PostMapping
    public Order create(@RequestBody Order order) {
        return repo.save(order);
    }
}
```

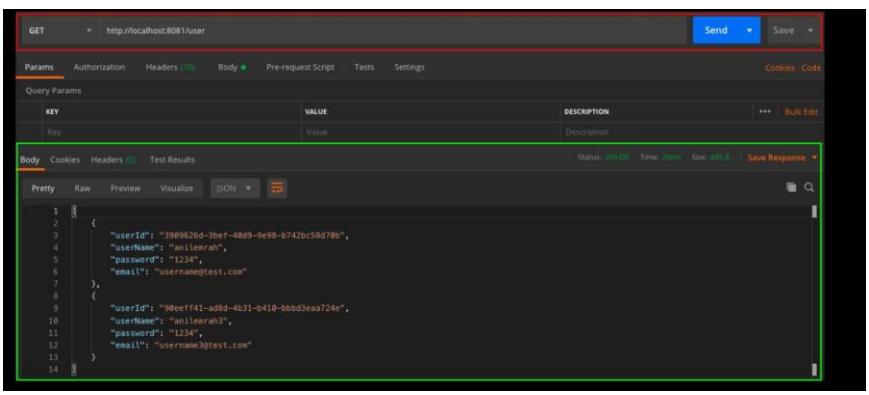
---

application.properties

server.port=8082
spring.datasource.url=jdbc:h2:mem:orderdb
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true

Output:





Result

Monolithic system successfully split into **two microservices** Each service: Runs independently Has separate responsibilities Can be deployed independently Achieved **loose coupling and scalability**